

Detection and Prevention of Network Attacks Using Snort

A Collaborative Machine-Learning-Assisted IDS/IPS Framework with Deception and Adaptive Response

Naqeebullah Niazi • Zafar Khan Armani

Department of Network Engineering, Faculty of Computer Science, Kabul Polytechnic University, Kabul, Afghanistan
Academic Supervisor: Abdul Shakoor "Azimi"

ABSTRACT

Network intrusion detection remains difficult when security teams must interpret high-volume alerts, correlate evidence across multiple observation points, and react before malicious activity causes service disruption. This paper presents a practical collaborative Snort-based IDS/IPS framework developed as a controlled laboratory prototype. The system integrates three Snort sensing nodes, centralized event correlation, an OpenCanary deception node, Wireshark validation, a Random Forest V3 behavior model, a Streamlit security dashboard, and distributed Linux packet-filtering actions. The machine-learning layer classifies behavior as ATTACK or NORMAL and supports three operational decisions - ALLOW, MONITOR, and BLOCK - using short-term event density, protocol, port category, source context, unique-port activity, credential-attempt frequency, and inter-event timing. The final behavior dataset contains 243 cleaned records (163 attack and 80 normal) and is evaluated with grouped out-of-fold validation by day. The model achieved 95.47% accuracy, 100% precision, 93.25% recall, and 96.51% F1-score, with no false positives in the grouped validation set. Live laboratory experiments verified coordinated responses to port scanning, SSH brute force, Telnet brute force, ICMP flooding, and HTTP/Web attack behavior. The framework also supports adaptive block duration, block extension, risk-based automatic unblocking, and independent handling of concurrent attackers. Comparison with recent Snort collaborative IDS, Snort-machine-learning, and SDN hybrid IDS/IPS studies shows that the principal contribution is not a claim of superior cross-dataset accuracy, but the end-to-end integration of collaborative sensing, behavior-aware machine learning, deception evidence, automated distributed prevention, recovery, and centralized operational visibility without relying on a SIEM platform.

Keywords - Snort, collaborative intrusion detection, intrusion prevention, Random Forest, OpenCanary, deception, machine learning, adaptive blocking, network security.

1. Introduction

Modern enterprise and academic networks are exposed to reconnaissance, credential attacks, flooding, and application-layer abuse. Signature-based network intrusion detection systems remain effective for known attack patterns, but a standalone sensor provides a limited view of distributed activity and can produce large volumes of alerts that require manual interpretation. Collaborative intrusion detection addresses this visibility problem by combining evidence from multiple sensors, while machine learning can add behavior-oriented scoring and prioritization. [1], [3]

The project reported in this paper originated from a bachelor monograph on detecting and preventing network attacks using Snort. Its practical objective was expanded from basic detection into a unified operational workflow: collect alerts from multiple Snort nodes, enrich them with deception evidence, evaluate behavior through a machine-learning layer, take an automated response when risk is high, and expose the complete incident state through a central real-time dashboard. The implementation is intentionally based on open-source and laboratory-accessible technologies, including Snort, OpenCanary, Wireshark, Python, scikit-learn, Streamlit, VMware Workstation, and EVE-NG.

The framework is evaluated as an engineering prototype rather than as a production-grade commercial security product. Its contribution therefore lies in practical integration and reproducibility. It does not use a Security Information and Event Management (SIEM) product; correlation, machine-learning decisions, mitigation state, and visualization are implemented directly in the project pipeline and dashboard.

1.1 Research Problem and Motivation

Three operational gaps motivated the design. First, an alert-only IDS may identify an event without preventing continued malicious traffic. Second, a single Snort node can miss context that becomes clearer when evidence is shared across sensors and a deception host. Third, a fixed response policy cannot easily distinguish an isolated benign event from dense attack behavior. These gaps suggest a pipeline in which detection, correlation, intelligent scoring, prevention, and recovery are treated as one continuous process.

1.2 Main Contributions

- A multi-sensor collaborative Snort architecture with centralized correlation and shared evidence from three sensing nodes.
- A behavior-based Random Forest V3 model using ten operational network features and grouped out-of-fold validation to reduce leakage from temporally related events.
- Integration of OpenCanary deception telemetry so credential and service-interaction evidence can strengthen attack interpretation.
- An adaptive response layer that maps model/behavior evidence to ALLOW, MONITOR, or BLOCK and performs distributed mitigation across defensive Linux nodes.
- Risk-based automatic recovery through timed re-evaluation, block extension when malicious behavior persists, and automatic unblocking when the risk decreases.
- A real-time Streamlit dashboard that joins Snort alerts, machine-learning decisions, deception evidence, distributed mitigation state, and incident reporting.
- Practical validation with both internal and external attack sources, including overlapping/concurrent attacker activity.

2. Related Work and Research Gap

2.1 Collaborative Snort IDS Nodes

Davies, Eiza, Shone, and Lyon proposed a 2025 Collaborative Intrusion Detection System built from multiple Snort IDS nodes connected to a central node and LogScale SIEM. Their testbed evaluates attacks including Nmap port scanning and ICMP flooding and emphasizes centralized correlation, reduction of alert fatigue, and improved visibility over standalone IDS deployment. Importantly, machine-learning integration is identified as a future enhancement rather than as an implemented component of the evaluated CIDS. [1]

The proposed framework shares the multi-Snort collaborative principle, but extends the operational loop by placing a Random Forest decision layer and automated distributed mitigation directly behind correlation. It also adds OpenCanary deception telemetry and risk-based recovery. Unlike the Davies architecture, the present implementation does not depend on SIEM software for correlation or visualization.

2.2 Collaborative IDS with a Snort Machine-Learning Plugin

Priambodo et al. combine Snort NIDS, Wazuh HIDS, Elastic Stack, Flask, and a custom machine-learning plugin. Their NSL-KDD experiments compare SVM, K-means, and neural networks. SVM achieves 97.7% training accuracy and 97.5% test accuracy; reported attack-specific results include 99.3% for DoS and 98.4% for Probe. Their real-time experiments also show that machine learning can provide more detailed interpretation than a predefined Snort rule set. [2]

This is the closest prior work in terms of combining collaborative intrusion detection and machine learning. The difference is architectural emphasis: the present work uses a smaller behavior-specific Random Forest dataset generated for the project, but expands the response plane through deception evidence, synchronized blocking/unblocking across multiple nodes, adaptive duration, and automatic risk re-evaluation.

2.3 Snort and Machine Learning Analysis

El Aeraj and Leghris analyze Snort architecture, signature-based limitations, and machine-learning approaches for improving detection. Their review summarizes prior Snort/ML studies in which SVM-based methods reached approximately 99% precision in selected experiments and reports examples with low false-positive rates. The study is primarily analytical and comparative; it argues for combining signature and learned behavior but does not implement the integrated multi-sensor prevention workflow evaluated here. [3]

2.4 SDN Hybrid IDS/IPS with Encrypted-Traffic Detection

Farahmand and Frotan present a hybrid IDS/IPS for Software-Defined Networking using rule-based detection, JA3 fingerprinting with a risk score, and Random Forest trained on CIC-IDS-2017. Their Ryu/Mininet implementation reports 97.26% overall IDS accuracy, 197 ms average IPS response, and 95.3% encrypted-traffic detection. This work is stronger in encrypted TLS visibility and formal latency measurement, while the present framework focuses on collaborative Snort sensing across independent Linux nodes, deception intelligence, adaptive mitigation, and recovery. [4]

2.5 Additional Relevant Research

Other studies reinforce individual elements of the design space. Shah and Issac compare open-source IDS performance and apply machine learning to Snort, supporting the value of an adaptive classification layer [7]. Shaukat et al. survey machine-learning techniques across cybersecurity and emphasize the importance of data quality, adaptability, and evaluation design [8]. Vadhil, Nanne, and Salihi discuss machine learning as a mechanism for improving open-source IDS accuracy [9]. Adiwal et al. demonstrate a Snort-based approach for DNS amplification and tunneling detection [10], while Tudosi proposes Python-based monitoring and troubleshooting for Snort in distributed environments [11]. Boukebous et al. compare Snort 3 and Suricata, highlighting the continuing relevance of performance and packet-processing efficiency in open-source NIDS deployments [12].

2.6 Research Gap

The literature contains strong solutions for collaborative sensing, Snort-plus-machine-learning classification, encrypted-traffic analysis, and centralized visualization. However, these capabilities are often evaluated separately or with a prevention plane that is not tightly integrated with deception and post-block re-evaluation. The gap addressed in this work is therefore an end-to-end laboratory architecture that joins multi-Snort collaboration, behavior-based machine learning, honeypot evidence, distributed prevention, adaptive blocking, automatic recovery, and one central operational view.

Table 1. Position of the proposed framework relative to selected prior work.

Study	Architecture	ML	Automated response	Distinctive emphasis
Davies et al. (2025)	Multi-Snort CIDS + central node + LogScale	Future work	Primarily detection / analyst response	Collaborative Snort and centralized correlation
Priambodo et al. (2024)	Snort NIDS + Wazuh HIDS + ELK/Flask	SVM (best)	Detection/classification oriented	NIDS-HIDS collaboration + Snort ML plugin
El Aeraj & Leghris (2024)	Snort/ML review and analysis	Reviews SVM and others	No integrated prevention prototype	Evidence for ML-enhanced Snort
Farahmand & Frotan	SDN/Ryu/Mininet hybrid IDS/IPS	Random Forest	Controller-based automatic IPS	JA3/TLS analysis + formal response latency
This work	3 Snort sensors + central pipeline + OpenCanary	Random Forest V3	Distributed adaptive block/unblock	Collaborative sensing + deception + prevention + recovery

3. Proposed System Architecture

The prototype combines a VMware security laboratory with an EVE-NG enterprise-style network. The VMware side contains the collaborative Snort sensors, central orchestration, and deception node. EVE-NG provides routed/switching infrastructure and an external Kali attack source, allowing the security pipeline to observe activity that originates both inside and outside the local sensor segment. This separation is useful for validating that the detection and response logic is based on observed source behavior rather than on a single hard-coded attacker.

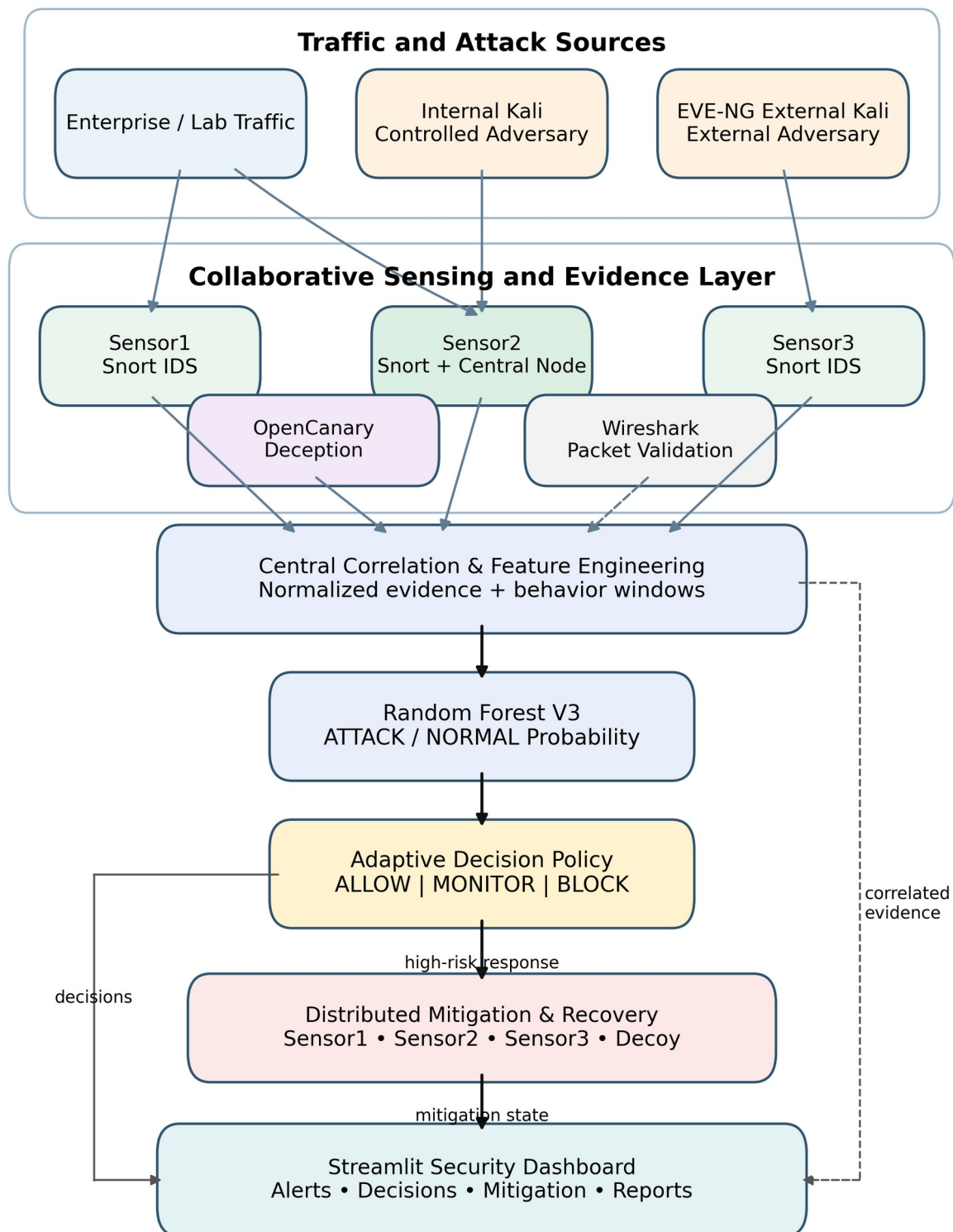


Figure 1. High-level architecture of the collaborative Snort/ML/deception framework.

3.1 Components and Roles

Table 2. Main testbed components and functional roles.

Component	Role	Main evidence / action
Sensor1	Distributed Snort sensing node	Network alerts; receives distributed mitigation state
Sensor2	Central Snort node and orchestration host	Correlation, ML runtime, state management, local mitigation, dashboard services
Sensor3	Distributed Snort sensing node	Network alerts; receives distributed mitigation state
OpenCanary Decoy	Deception / honeypot node	Credential attempts, service interaction, attacker evidence
Internal Kali	Controlled internal adversary	Port scans, brute force, flooding, web tests
EVE-NG external Kali	Controlled external adversary	External-source attack and concurrent-attacker validation
Wireshark	Packet-level validation tool	Independent packet evidence for analysis and troubleshooting
Streamlit dashboard	Central operational visualization	Alerts, AI/ML decisions, attack analytics, mitigation state, reports

3.2 Data Flow and Correlation

Snort alerts and OpenCanary events are normalized into a common operational record containing source address, protocol, destination port, attack context, and timestamp. The central runtime maintains short behavior windows for each source and attack type. This is important for concurrent attackers: one malicious source does not overwrite the state of another source, and a block is stored with its own source/reason/expiry state. A bounded batch reader prevents high-rate Snort floods from creating an unbounded processing backlog; model inference is aggregated by source and attack type within each batch while individual behavior events are still counted.

The dashboard is a direct project component rather than a SIEM front end. It reads project-generated alert, decision, collaborative, deception, and mitigation logs and presents them in a single view. This keeps the research focus on the behavior of the proposed pipeline itself.

4. Research Methodology

4.1 Experimental Approach

The study follows an applied experimental methodology. The system was first built and validated in a controlled virtual network; Snort rules and log paths were verified; OpenCanary was integrated for deception evidence; a behavior dataset was constructed and audited; the Random Forest model was trained and validated; and finally the model was inserted into the live correlation/response pipeline. Attack scenarios were then repeated while monitoring Snort logs, machine-learning decisions, distributed block state, and automatic recovery.

4.2 Behavior Dataset and Data Quality

The final Random Forest V3 dataset contains 243 records: 163 ATTACK and 80 NORMAL. Exact duplicate records were removed. The final model is intentionally trained on behavior features rather than on direct attack-name labels or other fields that would trivially reveal the target class. This design choice was made to avoid misleadingly high validation results caused by duplication or label leakage and to make the model reflect observable traffic behavior.

Table 3. Random Forest V3 behavior features.

Feature	Operational meaning
protocol_tcp	Binary indicator for TCP behavior
protocol_udp	Binary indicator for UDP behavior
protocol_icmp	Binary indicator for ICMP behavior
port_category	Coarse category of the target service/port

Feature	Operational meaning
src_internal	Whether the observed source is classified as internal
events_5s	Number of recent events in a 5-second window
events_20s	Number of recent events in a 20-second window
unique_ports_20s	Number of distinct destination ports in 20 seconds
credential_attempts_20s	Credential-related events in 20 seconds
time_since_previous	Elapsed time since the previous event from the source

4.3 Validation Design

The dataset is evaluated using grouped cross-validation by day and out-of-fold (OOF) predictions. Grouping by day is more conservative than randomly splitting highly related observations because events from the same activity period are less likely to appear in both training and validation folds. Accuracy, precision, recall, F1-score, and the confusion matrix are reported. The model remains binary (ATTACK/NORMAL); operational attack types are supplied by contextual parsing and deception evidence.

4.4 Live Decision Policy

The runtime combines the Random Forest attack probability with behavior density. Low-risk events are allowed, moderate-risk behavior is monitored, and high-risk or strongly suspicious behavior is blocked. Behavior overrides allow the system to escalate when event density, unique-port activity, or repeated credential attempts become strong even when the raw model action is initially only MONITOR. Block duration is risk-adaptive, and selected attack profiles can define a specific operational duration; for example, the current SSH brute-force policy uses a 180-second block before re-evaluation.

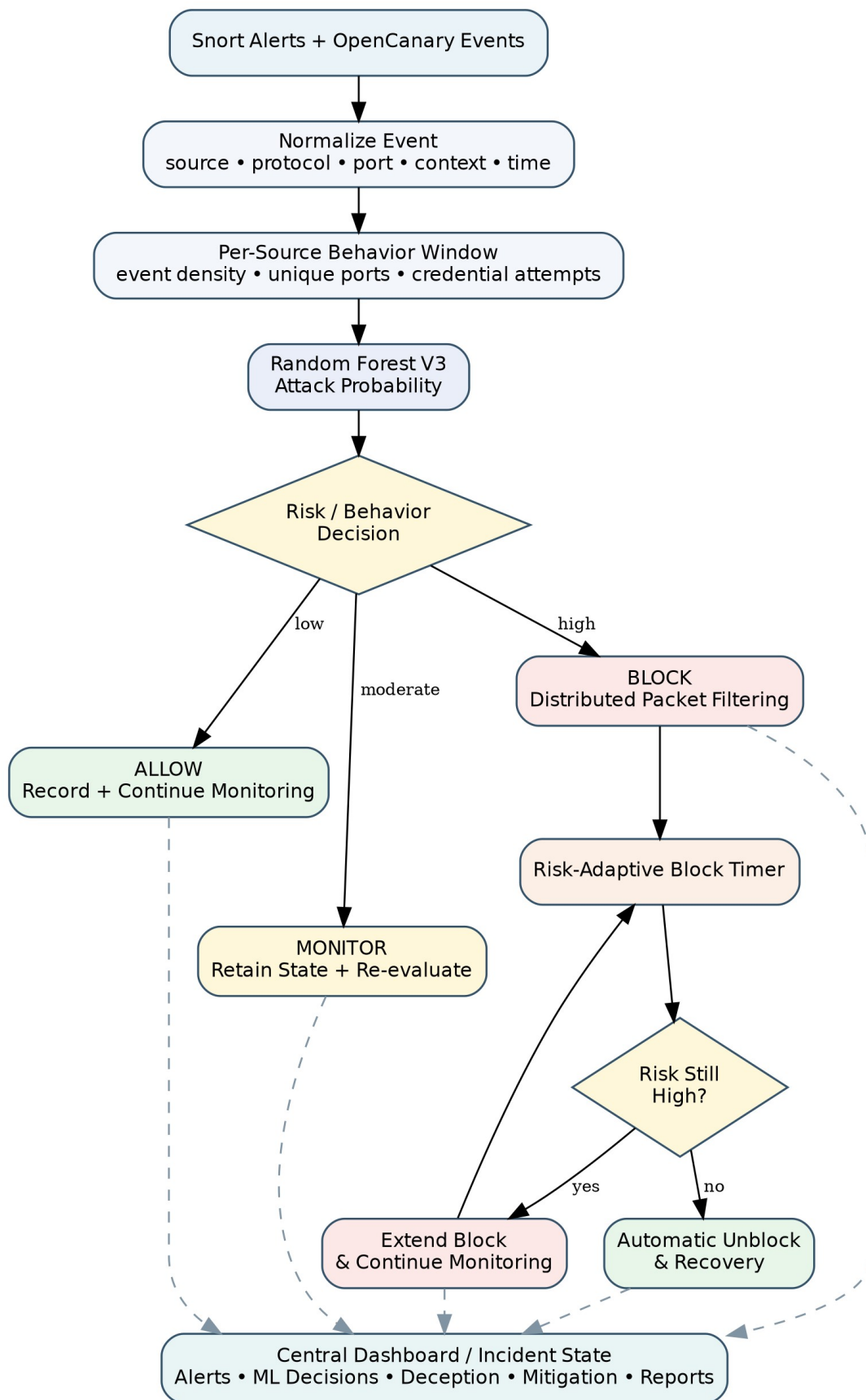


Figure 2. Runtime decision, distributed mitigation, and automatic recovery workflow.

4.5 Attack Scenarios

The final practical evaluation focuses on attack behaviors that are repeatable and observable in the controlled laboratory: port scanning, SSH brute force, Telnet brute force, ICMP flooding, and HTTP/Web interaction against the

decoy/application surface. Both single-attacker and overlapping multi-attacker scenarios are used. The objective is not exploitation of real external systems; all attack generation occurs against the project testbed.

5. Results

5.1 Random Forest V3 Validation

Grouped OOF validation produced 95.47% accuracy, 100% precision, 93.25% recall, and 96.51% F1-score. The confusion matrix contains 80 correctly classified NORMAL observations, no NORMAL observation incorrectly marked as ATTACK, 152 correctly classified ATTACK observations, and 11 attacks missed by the model. Therefore, the main validation weakness is false negatives rather than false positives.

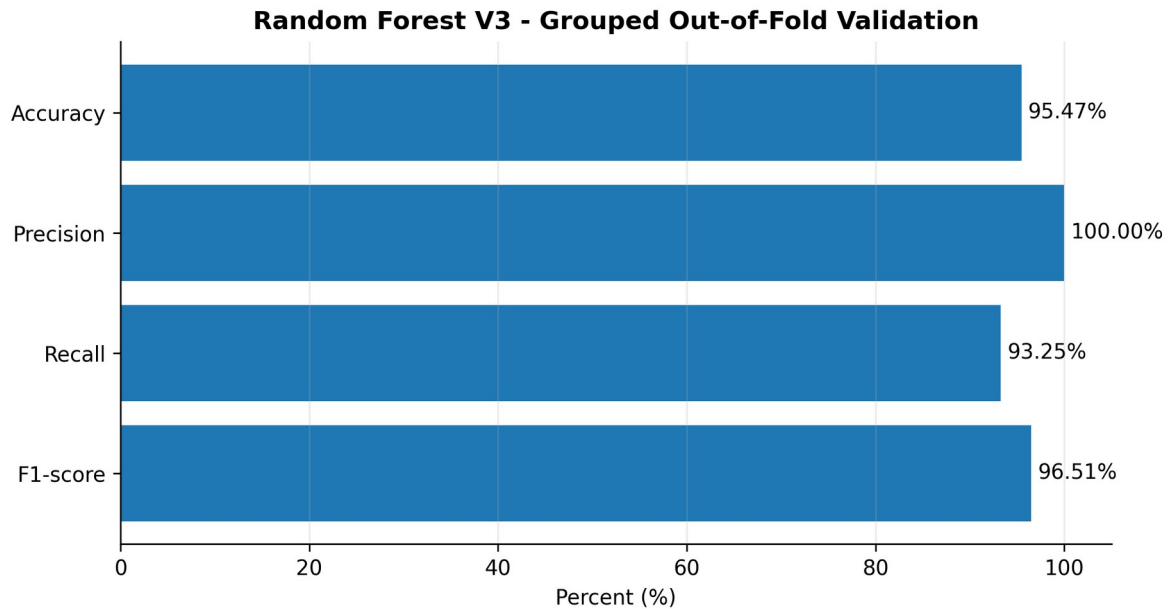


Figure 3. Grouped out-of-fold Random Forest V3 validation metrics.

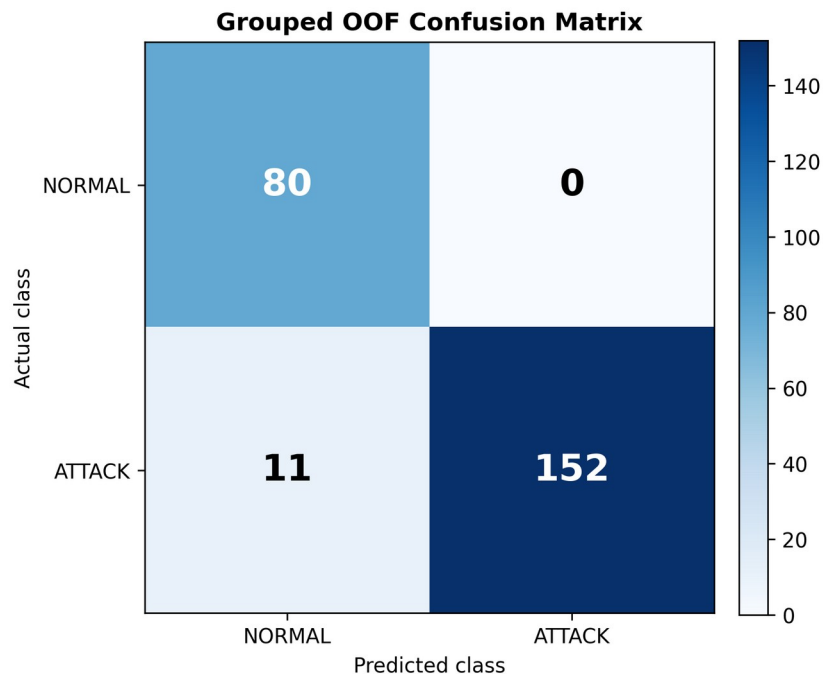


Figure 4. Grouped out-of-fold confusion matrix (rows = actual, columns = predicted).

Table 4. Random Forest V3 grouped OOF validation results.

Metric	Result
Accuracy	95.47%
Precision	100.00%

Metric	Result
Recall	93.25%
F1-score	96.51%
False positives	0
False negatives	11

5.2 Live Attack Response

The live results demonstrate that model probability and behavior policy are distinct but complementary. A port scan may begin with ALLOW or MONITOR decisions and later escalate to BLOCK when many unique ports are observed. Similarly, an ICMP flood can trigger an initial block, remain blocked through a risk-based extension when dense events persist, and later be automatically released after the behavior window decays.

Table 5. Representative live response evidence from the controlled laboratory.

Attack scenario	Observed risk / behavior	Response	Post-block outcome
Port scan	~81.09% with strong unique-port behavior	BLOCK; adaptive duration	Auto-unblock after final risk ~15.82%
SSH brute force	100% in the latest verified run	Distributed BLOCK for 180 s	Auto-unblock; final risk ~5.87%
Telnet brute force	Up to 100% with repeated credential behavior	Distributed BLOCK; adaptive policy	Auto-unblock; final risk ~5.87% in verified runs
ICMP flood	~97.23%; persistent flood can reach extension risk ~98.56%	BLOCK and possible extension	Auto-unblock after final risk ~34.58%
HTTP/Web attack	~78.42% after behavior escalation	BLOCK for representative 120 s run	Auto-unblock; final risk ~7.68%

5.3 Distributed Enforcement and Recovery

When the policy selects BLOCK, the source is inserted into the local Linux packet-filtering rules and synchronized to the other defensive nodes. The same source is later removed across the nodes when the expiry timer is reached and re-evaluation no longer supports continued blocking. This creates a full detect-to-recover chain instead of leaving mitigation as a manual operator task. Repeated laboratory verification confirmed distributed block and unblock behavior across Sensor1, Sensor2, Sensor3, and the Decoy node.

5.4 Concurrent Attacker Handling

A later robustness test used an internal attack source and an external EVE-NG Kali source during overlapping attack windows. The pipeline maintained independent source/reason state, allowing one source to be blocked for an ICMP-related event while another source was evaluated and mitigated for reconnaissance behavior. This validation is important because a central pipeline that stores only one active attacker or one global campaign state would fail when multiple sources are malicious at the same time.

5.5 Processing Robustness Under High-Rate Alerts

ICMP flooding exposed an engineering limitation in an earlier event loop: unbounded backlog consumption could keep old flood alerts in the inference path, making stale events appear recent and delaying automatic unblock. The runtime was revised to use bounded Snort batches, per-source/attack aggregation for inference, and bounded behavior history. After the change, backlog consumption completed, CPU load returned to a low steady level, and expired attackers were independently re-evaluated and unblocked. This result demonstrates the importance of pipeline design in addition to classifier accuracy.

6. Comparative Discussion

Cross-paper accuracy values must be interpreted cautiously. Priambodo et al. use NSL-KDD and report 97.5% SVM test accuracy, whereas Farahmand and Frotan use CIC-IDS-2017 in an SDN architecture and report 97.26% overall detection accuracy. The 95.47% result in this work comes from a much smaller, behavior-specific laboratory dataset evaluated with grouped OOF predictions. These values do not constitute a fair head-to-head ranking because the datasets, labels, attack distributions, feature spaces, and evaluation protocols are different. [2], [4]

A more defensible comparison is feature-level capability. Davies et al. demonstrate collaborative Snort correlation but identify ML as future work. Priambodo et al. implement ML within a Snort/Wazuh collaborative environment but concentrate mainly on detection and classification. El Aeraj and Leghris synthesize the evidence for ML-enhanced Snort. The SDN hybrid study adds JA3 encrypted-traffic analysis and formal sub-second IPS timing. The present framework contributes a different combination: independent Snort sensors, behavior ML, deception evidence, automated distributed response, adaptive duration, automatic unblock, and concurrent attacker state in one running pipeline.

Table 6. Feature-level comparison. “No” means not implemented as a central contribution in the supplied study.

Capability	Davies 2025	Priambodo 2024	El Aeraj 2024	Hybrid SDN	This work
Multi-Snort collaboration	Yes	Yes	Review	No	Yes
Live ML decision layer	Future	Yes (SVM)	Reviewed	Yes (RF)	Yes (RF V3)
Honeypot / deception evidence	No	No	No	No	Yes
Automated attack blocking	No central automation	Not central focus	No	Yes	Yes
Distributed block across defensive nodes	No	No	No	SDN controller	Yes
Adaptive block duration / extension	No	No	No	Fixed mechanisms	Yes
Risk-based automatic unblock	No	No	No	Fixed timeout	Yes
Encrypted TLS / JA3 analysis	No	No	Discussed limitation	Yes	No
Central real-time visualization	LogScale SIEM	Flask/ELK	No prototype	Flask/Socket.IO	Streamlit dashboard
Representative live tests	Port scan, ICMP flood	DoS, Probe	Reviewed cases	Floods, scans, web/TLS	Port scan, SSH/Telnet BF, ICMP, Web

6.1 Where the Proposed Framework Is Stronger

End-to-end operational integration: detection, correlation, learned risk, mitigation, recovery, and visualization are linked in one pipeline.

Deception-aware evidence: OpenCanary contributes credential/service interaction that is not available from Snort signatures alone.

Distributed mitigation and recovery: a malicious source can be blocked and later unblocked across several independent defensive nodes.

Behavior-sensitive escalation: the same type of event can remain ALLOW/MONITOR at low density and become BLOCK when behavior becomes attack-like.

Multi-attacker state: concurrent malicious sources are handled independently rather than through a single global attacker state.

6.2 Where Prior Work Is Stronger

The SDN hybrid study provides encrypted TLS/JA3 analysis, a feature not implemented in the current framework.

The SDN study also reports formal response-time, dashboard-delay, CPU/memory, and blocking-success measurements; the current project has not yet completed a comparable formal benchmark suite.

Priambodo et al. use a much larger established dataset and compare several machine-learning algorithms; the current V3 dataset is small and project-specific.

Davies et al. evaluate a larger multi-node CIDS design with dedicated database/SIEM infrastructure, while the current prototype prioritizes direct orchestration and laboratory simplicity.

7. Limitations and Future Work

The proposed system is a controlled laboratory prototype and should not be generalized to production networks without further validation. Its main limitations are the small size of the final behavior dataset, binary ATTACK/NORMAL model output, limited encrypted-traffic visibility, lack of a formal high-throughput benchmark

campaign, and reliance on a central orchestration node. The current design also assumes that collaborative sensors are trusted; compromised-sensor trust and signed event exchange are not yet addressed.

Collect a substantially larger multi-day, multi-network dataset and preserve strict temporal/source grouping during validation.

Move from binary ATTACK/NORMAL classification to a robust multi-class model while retaining contextual evidence for explainability.

Add JA3/JA4 or comparable encrypted-traffic fingerprinting and evaluate TLS/HTTPS threats without payload decryption.

Add anomaly/online learning for previously unseen behavior and carefully monitor concept drift and false positives.

Measure end-to-end detection latency, mitigation latency, throughput, packet loss, CPU, memory, and false-positive rate under sustained load.

Add central-node failover/redundancy and authenticated/signed sensor messages to reduce the central point of failure and improve trust.

Expand controlled attack coverage to DDoS, SQL injection, XSS, malware/ransomware-like behavior, and zero-day-like anomaly scenarios.

Integrate external threat-intelligence feeds and automated incident-response playbooks while keeping human oversight for high-impact actions.

8. Conclusion

This paper presented a practical Snort-based collaborative IDS/IPS framework that extends a bachelor network-security project beyond signature alerts into a complete detection-analysis-response-recovery workflow. Three Snort sensors provide distributed visibility; OpenCanary adds deception evidence; Random Forest V3 converts short-term behavior into ATTACK/NORMAL probabilities; the runtime converts risk into ALLOW, MONITOR, or BLOCK; distributed Linux packet filtering performs mitigation; timed re-evaluation supports extension or automatic unblock; and a Streamlit dashboard provides central operational visibility.

The final Random Forest V3 model achieved 95.47% grouped OOF accuracy, 100% precision, 93.25% recall, and 96.51% F1-score on the cleaned 243-record behavior dataset. Live experiments verified port-scan, SSH brute-force, Telnet brute-force, ICMP-flood, and HTTP/Web responses, including adaptive blocking and automatic recovery. Concurrent-attacker tests further showed that the framework can maintain independent source state. The comparison with recent research indicates that the central research contribution is architectural integration rather than numeric superiority: this work joins collaborative Snort sensing, machine learning, deception, distributed prevention, and recovery in a single laboratory system, while acknowledging that larger datasets, encrypted-traffic analysis, formal performance benchmarking, and stronger distributed trust remain important next steps.

References

- [1] T. Davies, M. H. Eiza, N. Shone, and R. Lyon, "A Collaborative Intrusion Detection System Using Snort IDS Nodes," 2025.
- [2] D. F. Priambodo, A. H. N. Faizi, F. D. Rahmawati, S. U. Sunaringtyas, J. Sidabutar, and T. Yulita, "Collaborative Intrusion Detection System with Snort Machine Learning Plugin," *International Journal on Informatics Visualization*, vol. 8, no. 3, pp. 1230-1238, Sep. 2024.
- [3] O. El Aeraj and C. Leghris, "Analysis of the SNORT intrusion detection system using machine learning," *International Journal of Information Science & Technology*, vol. 8, no. 1, Mar. 2024.
- [4] M. Farahmand and Z. H. Frotan, "Design and Simulation of a Hybrid IPS/IDS System based on SDN with Encrypted Traffic Detection Capability," Department of Network Engineering, Kabul Polytechnic University, research manuscript.
- [5] K. A. Scarfone and P. M. Mell, *Guide to Intrusion Detection and Prevention Systems (IDPS)*, NIST Special Publication 800-94, 2007.
- [6] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5-32, 2001.
- [7] S. A. R. Shah and B. Issac, "Performance comparison of intrusion detection systems and application of machine learning to Snort system," *Future Generation Computer Systems*, vol. 80, pp. 157-170, 2018.
- [8] K. Shaukat, S. Luo, V. Varadharajan, I. A. Hameed, and M. Xu, "A Survey on Machine Learning Techniques for Cyber Security in the Last Decade," *IEEE Access*, 2020.
- [9] F. A. Vadhil, M. F. Nanne, and M. L. Salihi, "Importance of Machine Learning Techniques to Improve the Open Source Intrusion Detection Systems," *Indonesian Journal of Electrical Engineering and Informatics*, vol. 9, no. 3, 2021.
- [10] S. Adiwal, B. Rajendran, D. Pushparaj, and S. Sudarsan, "DNS Intrusion Detection (DID) - A SNORT-based solution to detect DNS amplification and DNS tunneling attacks," *Franklin Open*, vol. 2, p. 100010, 2023.
- [11] A.-D. Tudosi, "A Python-Based Approach for Monitoring and Troubleshooting Snort IDS in Distributed Firewall Environments," in *International Conference on Advanced Scientific Computing (ICASC)*, 2023.
- [12] A. A. E. Boukebous, M. I. Fettache, G. Bendiab, and S. Shiaeles, "A Comparative Analysis of Snort 3 and Suricata," in *2023 IEEE IAS Global Conference on Emerging Technologies (GlobConET)*, 2023.
- [13] Thinkst Applied Research, *OpenCanary 0.9.5 Documentation*, 2024.
- [14] Wireshark Foundation, *Wireshark User's Guide*, 2024.
- [15] National Institute of Standards and Technology, *The NIST Cybersecurity Framework (CSF) 2.0*, 2024.

Research Integrity Note. All ACD-IPM model metrics, risk values, and attack outcomes reported here are from the controlled project laboratory. Metrics from prior work are reproduced only for contextual comparison. Because datasets, labels, architectures, and protocols differ, cross-study numbers should not be interpreted as directly equivalent benchmarks.